Автоматическая настройка гиперпараметров машинного обучения с помощью мета-управляемой байесовской оптимизации

Монсеф Гаруани, Мурад Бунеффа

Аннотация

Выбор одного или нескольких алгоритмов для оптимизирования машинного обучения (МО) и настройка значимых гиперпараметров являются одними из ключевых задач для продвинутого анализа данных с использованием методологий МО. Однако это одна из важнейших задач для применения решений на основе МО к решению реальных проблем. В этом отношении байесовская оптимизация (БО) является популярным методом для оптимизации функций типа "черного ящика". Но она всё ещё недостаточна для крупномасштабных проблем, поскольку не может использовать знания с прошлых применений. Основная сложность в этом аспекте связана вычислениями функции на плохих вариантах выбора гиперпараметров МО. Чтобы решить эту проблему, мы предлагаем интегрировать байесовскую оптимизацию с помощью мета-управления. Следовательно, мета-управляемая байесовская оптимизация предоставляет средства для использования знаний из предыдущих циклов оптимизации на схожих задачах. Эта возможность выступает в качестве предварительного условия для определения конкретных частей входного пространства, которые следует оценить следующим. В этой связи мы намерены направлять БО с помощью функционального дисперсионного анализа (ANOVA) конфигураций, предложенного процессом, мета-обучения. В этой статье мы демонстрируем на большой коллекции бенчмарков по оптимизации гиперпараметров, что предлагаемый подход с мета-управляемой оптимизацией примерно в 3 раза быстрее, чем стандартная БО. Тем самым он достигает нового уровня наивысшей производительности, что доказано экспериментами на 9 наборах данных для классификации.

Ключевые слова Оптимизация гиперпараметров · Байесовская оптимизация · Мета-обучение · Мета-управляемая байесовская оптимизация · fANOVA

1. Введение

Выбор подходящих алгоритмов машинного обучения и оптимизация связанных с ними гиперпараметров (ГП) являются повсеместными задачами в машинном обучении. Эти задачи, как правило, специфичны для рассматриваемой области исследований и реальных приложений [1]. Конфигурация гиперпараметров – это важный выбор проектирования для получения желаемой производительности алгоритмов МО, тогда как правильная настройка гиперпараметров может потребовать значительных затрат ценных ресурсов в плане времени и вычислительных мощностей, поскольку требует высокой технической экспертизы в области МО и математики, а также множества эмпирических решений [2]. В результате многие недавние исследовательские усилия в машинном обучении были сосредоточены на улучшении методов оптимизации гиперпараметров алгоритмов МО [1, 3-8].

Одним из самых эффективных методов оптимизации алгоритмов машинного обучения обычно признается последовательная оптимизация на основе модели, также называемая байесовской оптимизацией [4]. Эмпирические данные показали, что она дает лучшие результаты для настройки гиперпараметров сверточных сетей по сравнению с экспертами в предметной области, стабильно достигая более высоких результатов на бенчмарке CIFAR-10 при повторном тестировании [9]. Однако байесовская оптимизация определяется как общая структура функции оптимизации, и, как и любой другой общий метод оптимизации, она требует значительного количества оценок функции для выявления оптимальных областей высокой производительности при запуске на новой задаче оптимизации.

В то время как одни и те же алгоритмы МО с одним и тем же пространством гиперпараметров систематически оптимизируются для разных задач (наборов данных), эксперты-люди зачастую изучают производительность данного алгоритма на ранее виденных задачах, чтобы ознакомиться с его поведением относительно пространства ГП перед настройкой алгоритма для новой задачи, итеративно.

Этот подход всё еще может превосходить современные методы БО [1, 10]. Следуя этой методологии, мы утверждаем, что более выгодно выполнять трансферное обучение для глобальной оптимизации "черного ящика" в рамках байесовской оптимизации, чтобы сохранить проверенные возможности обобщения её базовой суррогатной модели.

Для достижения этой цели мы используем силу подходов метаобучения [1], которые изначально заимствуют наилучшие конфигурации гиперпараметров из схожих задач [10-12]. Эти методы позволяют нам использовать наилучшие конфигурации гиперпараметров из схожих задач и интегрировать эти знания в функцию приобретения, которая направляет процесс поиска на нахождение высококачественных решений. Поступая так, подходы метаобучения часто могут превосходить традиционные методы байесовской оптимизации [4].

Метаобучение или "обучение учиться" – это подраздел машинного обучения, который фокусируется на изучении характеристик задач, производительности алгоритмов МО и их взаимозависимостей [1]. В контексте оптимизации гиперпараметров это включает обучение метамодели, которая учится предсказывать наилучший набор гиперпараметров для данной задачи на основе предыдущего опыта со схожими задачами. Затем метамодель используется для управления процессом поиска для новой задачи, позволяя быстро находить хорошие решения без выполнения исчерпывающего поиска по всему пространству гиперпараметров. Интеграция метаобучения в функцию приобретения байесовской оптимизации является мощным подходом, который позволяет нам использовать знания, полученные из предыдущих задач, для управления процессом поиска для новой задачи [13]. Это мета-управление может помочь избежать распространенных проблем БО, таких как застревание в локальных оптимумах или трата вычислительных ресурсов на бесперспективные области пространства поиска. Внедряя метаобучение, мы можем достичь лучших результатов с меньшими вычислительными ресурсами, делая оптимизацию гиперпараметров более эффективной [13].

Остальная часть статьи структурирована следующим образом: в Разделе 2 формально определяется проблема оптимизации гиперпараметров и предоставляется теоретическая база по байесовской оптимизации, а также введение в функциональный дисперсионный анализ (ANOVA) для оценки важности гиперпараметров МО. Раздел 3 охватывает детали анализа важности гиперпараметров, включая используемые наборы данных, алгоритмы, гиперпараметры и функциональный дисперсионный анализ. Предлагаемый подход, мета-управляемая байесовская оптимизация, для ускорения процесса оптимизации гиперпараметров представлен в Разделе 4. Раздел 5 содержит комплексный набор экспериментов с 8 алгоритмами машинного обучения, демонстрирующих, что MGBO превосходит современный метод БО, находя хорошие настройки гиперпараметров в 2-3 раза быстрее. Наконец, Раздел 6 завершает статью и предоставляет план будущих исследований.

2. Теоретическая база и связанные работы

В этом разделе мы исследуем теоретический фундамент и две тесно связанные исследовательские области: (1) байесовскую оптимизацию для оптимизации гиперпараметров МО и (2) использование функционального дисперсионного анализа для оценки значимости гиперпараметров. Кроме того, мы исследуем недостатки классической байесовской оптимизации, чтобы обосновать предлагаемый подход.

2.1 Оптимизация гиперпараметров

Задача выбора алгоритма или семейства алгоритмов, которые могут показать лучшую производительность на данной комбинации наборов данных и метрик оценки, является важной задачей в машинном обучении [11, 14-17]. Алгоритмы машинного обучения обычно имеют два типа параметров: обычные параметры, которые модель изучает и оптимизирует автоматически в процессе обучения на основе регулярного поведения модели, и гиперпараметры, которые задаются вручную перед обучением модели и могут быть категориальными, непрерывными или условными (см. Таблицу 1). Выбор подходящих гиперпараметров может быть критически важен для достижения оптимальной производительности модели. Этот процесс, известный как настройка гиперпараметров, включает

поиск оптимальных значений для этих гиперпараметров, которые приводят к наилучшей производительности модели.

ML algorithm	Nb, of hyperparameters	Nb, of ordinary parameters
Decision Tree	3	1
SVM	5	2
Logistic regression	6	4
Random forest	4	2

Таблица 1 Пространство конфигураций некоторых алгоритмов классификации

С теоретической точки зрения, для выбора оптимальных значений гиперпараметров необходимо тщательно исследовать все возможные комбинации ГП. Однако количество и разнообразие ГП делают эту задачу чрезвычайно сложной и потенциально неосуществимой. Риск комбинаторного взрыва возрастает с увеличением количества ГП и их возможных значений. Это затрудняет определение наилучшей возможной комбинации ГП для максимизации производительности модели. В результате поиск оптимальных ГП требует тщательного баланса между эффективностью поиска и точностью. Существует множество техник и инструментов для автоматизации процесса выбора ГП, включая поиск по сетке, случайный поиск, байесовскую оптимизацию и генетические алгоритмы [18]. Тем не менее, задача остается в том, чтобы найти наиболее эффективный подход для выбора оптимальных ГП для данной модели [19].

Чтобы прояснить далее, давайте определим $\mathcal{H}$ как множество конфигураций гиперпараметров для алгоритмов машинного обучения A, и $A(H)$ как модель, индуцированную алгоритмами МО A с конфигурацией ГП $H \in \mathcal{H}$. В традиционных настройках машинного обучения проблема минимизации функции потерь $\mathcal{L}(A(H), \mathcal{D})$, оценивающей прогностическую производительность $A(H)$ на заданном наборе данных $\mathcal{D}$, рассматривается как задача "черного ящика" (Ур. 1).

Это означает, что единственный способ взаимодействия с целевой функцией $\mathcal{L}$ – это оценивать её для различных конфигураций гиперпараметров $H \in \mathcal{H}$. Однако, поскольку отдельные оценки $A(H)$ на реальных наборах данных могут занимать дни или даже недели, количество возможных оценок ограничено, что может повлиять на качество найденных наилучших конфигураций ГП [5, 6].

$$\mathcal{L}(A(H), \mathcal{D}) \equiv \{\, \mathcal{L}(A(H), \mathcal{D}) \mid \exists H \in \mathcal{H} \subseteq \perp \,\} \qquad (1)$$

За последнее десятилетие для оптимизации алгоритмов машинного обучения применялся ряд методов настройки гиперпараметров [10, 12, 20, 21]. Некоторые из этих методов включают итеративное построение множества $\mathcal{H}$ настроек гиперпараметров и вычисление функции потерь $\mathcal{L}(A(H), \mathcal{D})$ для каждой конфигурации алгоритма A на наборе данных $\mathcal{D}$. Это позволяет одновременно исследовать различные области пространства поиска гиперпараметров. Существуют различные популяционные стратегии настройки ГП, каждая со своим уникальным подходом к эволюции $\mathcal{H}$ на каждой итерации [4]. Наша цель в этом контексте – определить оптимальную конфигурацию ГП H^+, которая максимизирует функцию потерь заданного алгоритма МО A на наборе данных $\mathcal{D}$ в пространстве гиперпараметров $\mathcal{H}$, как показано ниже:

$$\mathcal{L}(A(H^+), \mathcal{D}) \equiv \{\, \mathcal{L}(A(H^+), \mathcal{D}) \mid \exists H^+ \in \mathcal{L}(A(H), \mathcal{D}) \ \cap \ H \in \mathcal{H} \,\} \qquad (2)$$

Аналогично, конфигурация гиперпараметров с наилучшей конфигурацией H^-, которая минимизирует функцию потерь алгоритма машинного обучения A на наборе данных $\mathcal{D}$, учитывая его пространство гиперпараметров $\mathcal{H}$, выглядит следующим образом:

$$\mathcal{L}(A(H^-), \mathcal{D}) \equiv \{\, \mathcal{L}(A(H^-), \mathcal{D}) \mid \exists H^- \in \mathcal{L}(A(H), \mathcal{D}) \ \cap \ H \in \mathcal{H} \,\} \qquad (3)$$

Для решения этой задачи самыми простыми и часто используемыми являются подходы Grid Search (поиск по сетке) и Random Search (случайный поиск). Первый более подходит для задач малой размерности, например, когда нужно настроить немного ГП. Для более сложных сценариев GS не способен исследовать более узкие перспективные области из-за большого гиперпространства [22]. Второй способен исследовать любое возможное решение гиперпространства, но лишен направленного

поиска, что приводит к высоким вычислительным затратам [2]. Метаэвристики, такие как генетические алгоритмы [23], алгоритмы оценки распределения [19] и последовательная оптимизация на основе модели [4], исследовались для настройки гиперпараметров из-за их вероятностной природы и способности выполнять направленный поиск. Однако этим техникам может потребоваться оценка большого количества кандидатных конфигураций, что приводит к высоким вычислительным затратам и времени сходимости; таким образом, необходимы дальнейшие исследования, чтобы исключить итеративную оценку функции на менее производительных конфигурациях для повышения эффективности методов настройки гиперпараметров МО [24].

2.2 Байесовская оптимизация

Байесовская оптимизация [25], также известная как последовательная оптимизация на основе модели, – это подход для оптимизации функций типа "черного ящика", таких как: $f: X \to \mathbb{R}$, оценка которых на заданном входном пространстве $\mathcal{X}$ является затратной. Байесовская оптимизация аппроксимирует H^+/H^- с помощью итеративных подгонок так называемой суррогатной модели, в дальнейшем обозначаемой как Ψ, на множестве наблюдаемых значений, обозначаемых как Γ, функции потерь $\mathcal{L}$, зависящей от предыдущих значений функции, представленных как $y_1, y_2, \dots, y_n$, относительно $H_1, H_2, \dots, H_n$. Множество наблюдаемых значений можно формально представить следующим образом:

$$\Gamma \equiv \{(H_1, \mathcal{L}(A(H_1), \mathcal{D})), \dots, (H_n, \mathcal{L}(A(H_n), \mathcal{D}))\} \quad (4)$$

Алгоритм байесовской оптимизации требует двух строительных блоков, приведенных ниже:

1. **Байесовский априор**, который задает априорное представление о возможных целевых функциях.
2. **Байесовский апостериор**, который предоставляет механизм для последовательного обновления представления путем обучения на новых наблюдениях.

Байесовская оптимизация достигает этой цели, строя байесовский апостериор для $\mathcal{L}$ на основе множества оцененных гиперпараметров. На каждой итерации выбирается и оценивается новая конфигурация на основе байесовского апостериора. Последний обновляется, чтобы включить новый ГП (H_{t+1}), с использованием вероятностной суррогатной модели, обусловленной текущей историей оптимизации H_t. Обычно в качестве суррогатной модели используется гауссовский процесс (GP), и в этом случае результирующее апостериорное представление $f(x)$ следует гауссовскому распределению со средним $\mu_t(x)$ и дисперсией $\sigma_t^2(x)$, как показано ниже:

$$\begin{aligned} \mu_t(x) &= \mathbb{E}\{f(x) \mid H_t \in \mathcal{H}\} \\ \sigma_t^2(x) &= \mathbb{V}\{f(x) \mid H_t \in \mathcal{H}\} \end{aligned} \quad (5)$$

Точки, исследуемые алгоритмом байесовской оптимизации, определяются **Функцией Приобретения (Acquisition Function, AF)**, которая присваивает значение полезности каждому $H \in \mathcal{H}$. Значение полезности определяется балансом между прогнозируемым значением и неопределенностью прогноза для каждого H. Этот итеративный процесс повторяется до достижения предопределенного бюджета испытаний или пока производительность кросс-валидации не достигнет удовлетворительного уровня. Алгоритм 1 предоставляет представление этого процесса.

Эффективность результирующего оптимизатора в значительной степени основана на тщательном проектировании функции приобретения [20]. В этой работе мы использовали **Ожидаемое Улучшение (Expected Improvement, EI)** [26] в качестве функции приобретения, которое количественно определяет ожидаемое улучшение по сравнению с наилучшим найденным на данный момент значением функции (Ур. (7)). Альтернативами EI могут быть Вероятность Улучшения [27], Верхние Доверительные Границы [28], методы на основе энтропии [29] и градиент знания [30]. Каждый из этих методов имеет свои преимущества и ограничения, что делает выбор функции приобретения зависимым от конкретной задачи оптимизации. Например, Вероятность Улучшения благоприятствует

эксплуатации областей с более высокими вероятностями нахождения глобального максимума, в то время как Верхние Доверительные Границы подчеркивают исследование областей с высокой неопределенностью. Методы на основе энтропии отдают приоритет получению информации о неизвестной функции, в то время как градиент знания стремится максимизировать прирост информации при следующем наборе оценок функции.

$$EI_t(x) \equiv \int_{-\infty}^{+\infty} max(y^+ - y),0)p(y \mid x)dy(7)$$

где текущее значение потерь, обозначаемое y^+, представляет собой наилучшее значение среднеквадратической ошибки (RMSE), найденное на данный момент. Функция плотности вероятности $p(y \mid x)$ моделирует неопределенность целевой функции в точке x и может быть представлена вероятностной моделью, такой как гауссовский процесс (GP). Член $max(y^+ - y, 0)$ гарантирует, что функция EI учитывает только улучшения по сравнению с текущим наилучшим наблюдением.

Следовательно, выбор адекватной функции приобретения играет решающую роль в байесовской оптимизации, поскольку он определяет, как оптимизатор балансирует исследование и эксплуатацию для сходимости к оптимальному решению (конфигурации гиперпараметров). В нашем исследовании мы обнаружили, что ожидаемое улучшение было наиболее подходящей функцией приобретения для нашей задачи оптимизации, что привело к улучшенной производительности по сравнению с другими протестированными методами.

2.3 Функциональный дисперсионный анализ (fANOVA) для оценки важности гиперпараметров МО

Функциональный дисперсионный анализ, или сокращенно fANOVA, – это статистическая техника, используемая для оценки важности гиперпараметров в алгоритмах машинного обучения [31]. Она особенно полезна для отбора и настройки гиперпараметров, поскольку дает представление об относительной важности различных гиперпараметров в определении производительности заданного алгоритма [13]. Основная идея, лежащая в основе функционального ANOVA, заключается в разложении вариации производительности модели на вклады от отдельных гиперпараметров, а также их взаимодействий. Это позволяет определить, какие гиперпараметры наиболее важны для достижения высокой производительности, и понять, как они взаимодействуют друг с другом [31].

В fANOVA пространство гиперпараметров делится на подмножества, каждое из которых состоит из одного гиперпараметра. Затем влияние каждого гиперпараметра на производительность алгоритма оценивается путем построения модели, описывающей взаимосвязь между гиперпараметром и метрикой производительности [32]. Эта модель строится путем подгонки функции к наблюдаемым точкам данных и разложения её на составляющие части, каждая из которых представляет эффект одного гиперпараметра. Анализируя результирующее разложение, можно определить, какие гиперпараметры оказывают наибольшее влияние на производительность алгоритма [31]. Эта информация затем может быть использована для руководства процессом оптимизации гиперпараметров, сосредотачиваясь на наиболее важных гиперпараметрах для достижения наилучшей возможной производительности [13].

Функциональный ANOVA может быть применен при наличии:

- Алгоритма $A \in \mathcal{A}$ с n гиперпараметрами в пространстве гиперпараметров $\mathcal{H}$
- Большого количества наборов данных $\mathcal{D}_1, \dots, \mathcal{D}_m$, где m – количество наборов данных (в нашем исследовании $m = 400$)
- Для каждого из наборов данных – набор эмпирических измерений производительности $\langle H_n, y_i \rangle_{i=1}^{k}$ для различных настроек гиперпараметров $H_n \in \mathcal{H}$, где y_i – производительность алгоритма A, измеренная рассматриваемыми метриками

производительности (т.е. Accuracy, Precision, Recall, F1 score). **Маргинальная производительность** $\hat{a}_n(H_n)$ определяется как средняя производительность алгоритма A для всех полных конфигураций H_n, имеющих в общем пространстве ГП $\mathcal{H}$.

Сначала мы собираем данные о производительности $\langle H_n, y_i \rangle_{i=1}^{k}$ для каждого из рассматриваемых алгоритмов с $k = 1000$ различных конфигураций ГП. Затем мы подгоняем модель случайного леса к данным о производительности и используем функциональный ANOVA для декомпозиции дисперсии в производительности случайного леса

Algorithm 1 Pseudo-code of the Sequential Model-based Optimization.

Input: Hyperparameters space $\mathcal{H}$, observation history Γ, number of iterations T, acquisition function AF, surrogate model Ψ.
Output: Best hyperparameters configuration found.
$\mathcal{L}^+ \leftarrow \emptyset$ the best value found so far
for t=1toT **do**
  Fit Ψ on Γ
  $H \leftarrow \underset{x \in X}{\operatorname{argmax}}\, AF((\Psi(H_t), \mathcal{L}^+)$
  Evaluate f(H) Expensive step
  $\Gamma \leftarrow \Gamma \cup \{(H, f(H))\}$
  If $f(H) < \mathcal{L}^+$ **Then**
    $H^+, \mathcal{L}^+ \leftarrow H, f(H)$
return H^+

$\hat{\mathbf{y}}: H_1 \times H_2 \ldots H_n \to \mathbb{R}$ на аддитивные компоненты, которые зависят от подмножеств гиперпараметров H_n:

$$\hat{\mathbf{y}}(H) = \sum_{u \in \mathcal{H}} \hat{f}_u\,(H_u) \quad (8)$$

где компонента $\hat{f}_u(H_u)$ определяется следующим образом:

$$\hat{f}_u(H_u) = \begin{cases} \hat{f}_\theta & \text{если } u = \theta \\ \hat{a}_u(H_u) - \sum_{w \in u} \hat{f}_w\,(H_w) & \text{иначе} \end{cases} \quad (9)$$

где $\hat{f}_\theta$ – среднее значение функции $\hat{\mathbf{y}}$ по её области определения. Унарная функция $\hat{f}_j(H_j)$ отражает важность гиперпараметра H_j в среднем по всем возможным значениям остальных гиперпараметров, в то время как $\hat{f}_u(H_u)$ отражает эффекты взаимодействия между всеми гиперпараметрами в $\mathcal{H}$. Функциональный ANOVA раскладывает дисперсию $\mathbb{V}$ в $\hat{\mathbf{y}}$ на вклады $\mathbb{V}_u$ всех возможных подмножеств гиперпараметров H_u алгоритма $\mathcal{A}$.

$$\mathbb{V} = \sum_{u \in n} \mathbb{V}_u \text{ ,Где} \mathbb{V}_u = \frac{1}{\| H_u \|} \int \hat{f}_u(H_u)^2 \mathrm{d}H_u \quad (10)$$

Важность гиперпараметра или набора гиперпараметров отражается долей дисперсии, за которую ответственен гиперпараметр или набор гиперпараметров. Чем выше доля, тем важнее гиперпараметр или набор гиперпараметров для модели. Таким образом, такой гиперпараметр следует дополнительно настраивать для достижения хорошей производительности.

3. Важность гиперпараметров алгоритмов МО на различных наборах данных

Наше исследование включало проведение FANOVA на 400 наборах данных с использованием 08 распространенных классификаторов для обучения с учителем. Мы исследуем метод опорных векторов (SVM), случайный лес, Extra Trees, Adaboost, деревья решений, логистическую регрессию, SGD и

градиентный бустинг. Мы использовали scikit-learn для реализации этих классификаторов и установили гиперпараметры и диапазоны, используемые в исследовании, которые подробно описаны в Таблице 2. Для стандартизации наборов данных мы применили согласованные шаги предварительной обработки данных ко всем обучающим наборам данных. Пропущенные значения заполнялись с использованием моды для категориальных признаков. Для числовых признаков стратегия заполнения была одним из гиперпараметров. Кроме того, категориальные гиперпараметры были преобразованы с помощью one-hot-encoding, а постоянные признаки были удалены для обеспечения единообразия нашего анализа.

Algorithm	Hyperparameter	Values
SVM	Degree	[2, 3]
	complexity(or:‘C’)	[1e−10,500] (log-scale)
	coef()	[0.,10]
	gamma	[1e−3,1.01] (log-scale)
	Kernel	'poly', 'rbf'
		[1, 20]
RandomForest&ExtraTrees	Min_samples_leaf	
	Min_samples_split	[2, 20]
	Bootstrap	{true, false}
	Imputation	{mean, median, mode}
	Max_features	[0.1,0.9]
	splitcriterion	{entropy, gini}
Adaboost	Algorithm	{SAMME, SAMME.R}
	Learningrate	[0.01,2.0] (log-scale)
	Max_depth	[1, 11]
	N_estimators	[50, 501]
DecisionTrees	Criterion	[‘entropy’, ‘gini’]
	Max features	[0.1, 0.9]
	Min_samples_leaf	[1, 21]
	Min_samples_split	[2, 21]
LogisticRegression	C	[1e−10,10.] (log-scale)
	Penalty	{’l2’, ’l1’}
	Fit_intercept	{True, False}
SGD	Power_t	[0.,5.]
	Eta()	[0.,5.]
	Learning rate	{’constant’,’optimal’,’invscaling’}
	Loss	{’hinge’, ’perceptron’, ’log’}
	penalty	{’l2’,’l1’,’elasticnet’}
GradientBoosting	Learning_rate	[0.01,1]
	Criterion	{’friedman_mse’, ’mse’}
	N_estimators	[50, 501]
	Max depth	[1, 11]
	Min_samples_split	[2, 21]

Table2 Hyperparameters tuned in the experiments

Для каждого из 400 наборов данных, ранее изученных в [1, 18, 33], мы собрали данные о производительности для 08 алгоритмов на 400 наборах данных, запустив не менее 1000 различных конфигураций гиперпараметров для каждого алгоритма на каждом наборе данных на мощном вычислительном кластере. Этот процесс привел к созданию базы знаний (KB) из 04 миллионов запусков моделей. Для обеспечения статистической достоверности мы убедились, что каждый набор данных имел не менее 8000 запусков, чтобы модель функционального ANOVA была достаточно надежной для оценки важности гиперпараметров алгоритмов МО на различных наборах данных.

Обзор процесса построения KB показан в алгоритме 2. Подробное описание используемых наборов данных можно найти в [18].

Для каждого набора данных мы используем функциональный ANOVA для декомпозиции дисперсии в производительности каждого классификатора МО и анализа вклада каждого гиперпараметра и их взаимодействий. Агрегируя вклады дисперсии по всем наборам данных, мы смогли определить гиперпараметры, которые оказывают более значительное влияние на производительность. Чтобы предоставить визуальное представление этого анализа, на Рис. 1 представлены графики типа "скрипка" распределения важности для некоторых гиперпараметров алгоритмов.

Изучая графики, мы можем определить, какие гиперпараметры оказывают наибольшее влияние на метрику производительности, так как они будут иметь более высокое медианное значение важности. Эта информация используется для руководства выбором и настройкой гиперпараметров в последующих экспериментах, что приводит к улучшению общей производительности.

Algorithm2 The knowledge base construction process.

```
1: Input: ClassAlgs[..],                                  available classification algorithms
     HPSpace[..],                                         set of HPs configurations to be applied
     PerfMeasures[..]                                     set of performance measures to acquire
2: Output: KB[#measure][#metadata]                        meta-knowledgebase
3: function CreateMetaKB(datasets[])
4:    metadata[]=∅
5:    for each measure in PerfMeasures do
6:       for each dataset D in datasets do
7:          DMetaFeatures = ComputeMetaFeatures(D);
8:          for each algorithm A in ClassAlgs do
9:             for each hyperparameters_configuration H in HPSpace do
10:               DPerfMeasure = GetPerformanceWith5FoldCV(A, H, D);
11:               metadata[]←DMetaFeaturesUDPerfMeasure;
12:      meta_ds[measure]←metadata[];
```

Ось X отображает изучаемый гиперпараметр, в то время как каждая точка данных соответствует значению $\forall i/\forall$, связанному с этим гиперпараметром (ур. 10). Более высокое значение указывает, что этот гиперпараметр вносит значительный вклад в дисперсию и может привести к существенной потере точности, если не настроен должным образом. Рисунок 1 иллюстрирует, что для классификатора Adaboost гиперпараметры максимальная глубина и скорость обучения были основными ГП, влияющими на производительность классификатора почти на всех наборах данных, в то время как gamma и C были наиболее значимыми для классификатора SVM.

Результаты этого анализа не предполагают, что настройка только упомянутых гиперпараметров достаточна. Напротив, когда доступен достаточный бюджет, все же рекомендуется настраивать все гиперпараметры. Однако, как было показано в предыдущем исследовании [32], концентрация на настройке наиболее важных гиперпараметров приводит к более быстрой сходимости. Поэтому в ситуациях, когда бюджет на настройку ограничен, может быть целесообразно расставить приоритеты в настройке наиболее важных гиперпараметров. Чтобы проверить эту гипотезу, предлагается мета-управляемая байесовская оптимизация, в которой процедура настройки изначально концентрируется на наиболее значимых гиперпараметрах и постепенно включает менее важные.

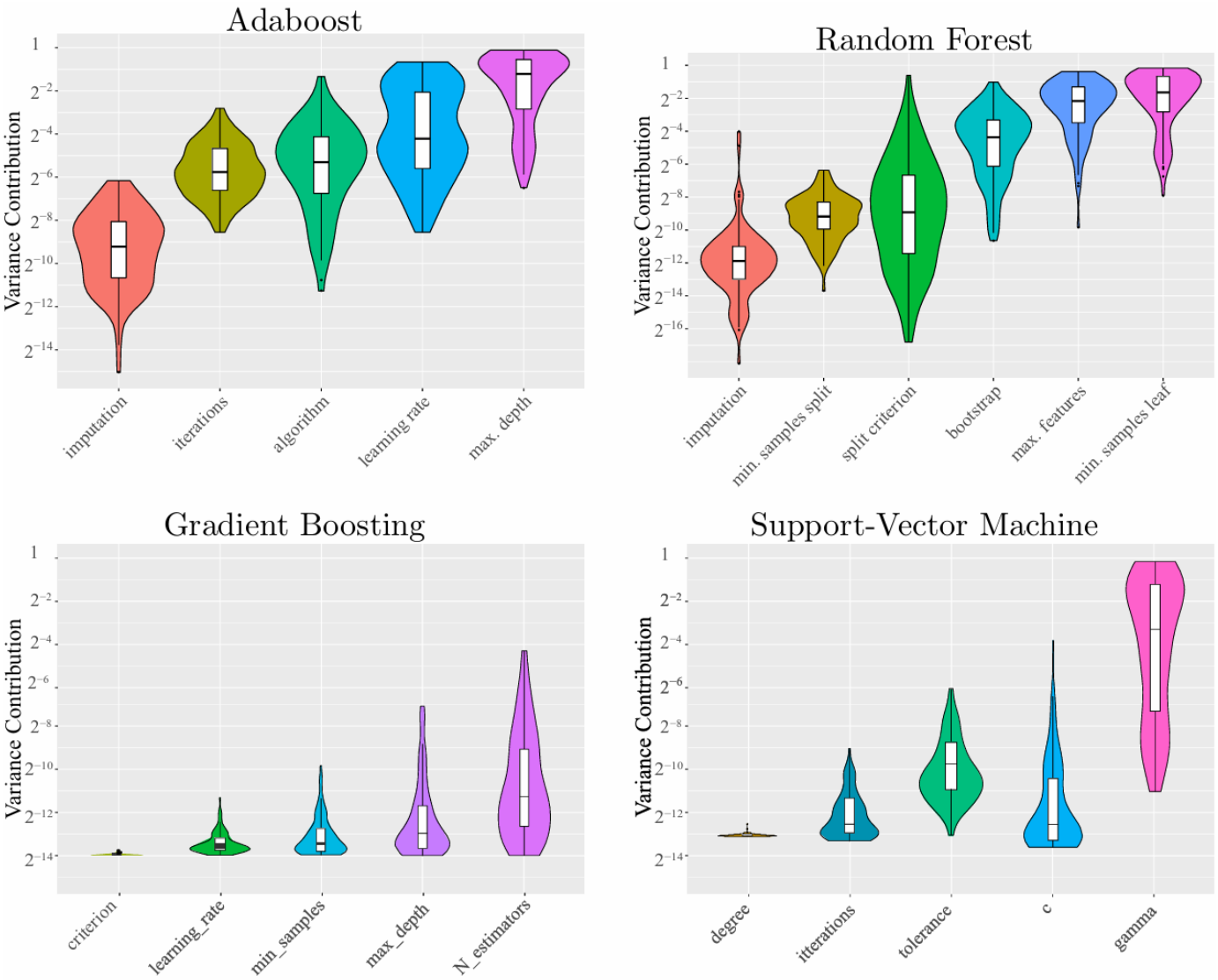

Fig. 1 Marginal contribution per 400 datasets for analyzed classifiers

4. Мета-управляемая байесовская оптимизация

В этом разделе мы представляем предлагаемый подход **Мета-управляемой байесовской оптимизации (MGBO)**. Алгоритм MGBO интегрирует метаобучение в алгоритм байесовской оптимизации для повышения эффективности и результативности процесса настройки гиперпараметров. В частности, MGBO включает fANOVA для определения важности гиперпараметров различных алгоритмов машинного обучения. Эта мета-управляющая модель используется для направления поиска оптимальных гиперпараметров в процессе байесовской оптимизации. Рисунок 2 дает глобальную архитектуру предлагаемого рабочего процесса мета-управляемой байесовской оптимизации.

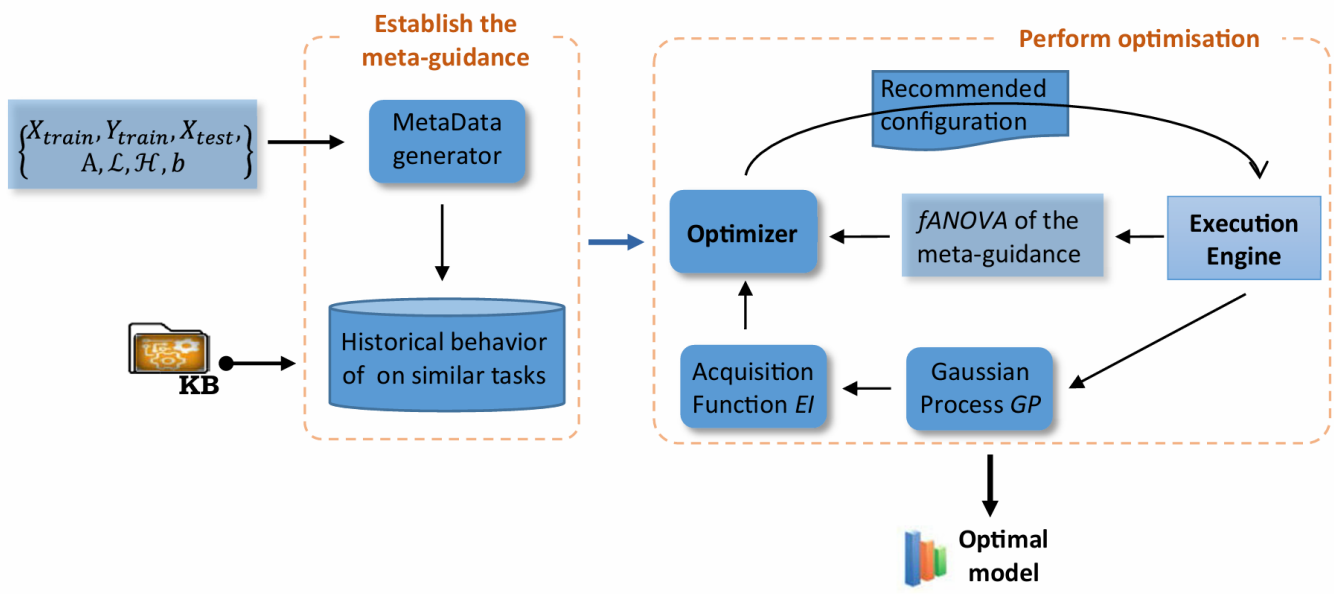

Fig. 2 Workflow of the Meta-Guided Bayesian Optimization(MGBO)

Подход MGBO выполняется в два этапа. На первом этапе проводится процесс метаобучения [1] на множестве из 400 наборов данных для улавливания взаимосвязей между характеристиками наборов данных и оптимальными гиперпараметрами для различных моделей машинного обучения, чтобы получить оптимальные алгоритмы МО для начала работы [1]. На втором этапе fANOVA выбранного алгоритма используется для направления процесса байесовской оптимизации для нового набора данных, предлагая набор начальных гиперпараметров, которые, вероятно, будут хорошо работать на заданном наборе данных. Затем процесс байесовской оптимизации итеративно уточняет предложенные гиперпараметры, оценивая их производительность на заданном наборе данных, пока не будут найдены оптимальные значения гиперпараметров.

Используя fANOVA для направления процесса поиска, MGBO сокращает количество конфигураций гиперпараметров, которые необходимо оценить на целевом наборе данных, что приводит к значительной экономии времени и вычислительных ресурсов. Сначала мы описываем суррогатную модель байесовской оптимизации, используемую в нашей структуре, прежде чем изложить процесс выбора следующей исследуемой конфигурации гиперпараметров.

Гауссовский процесс (Gaussian Process, GP) [34] – популярный выбор из-за его выдающихся особенностей, таких как поддержка зашумленных наблюдений и возможность использования методов на основе градиента [35]. Стандартный GP в байесовской оптимизации непосредственно моделирует $p(y \mid x)$ для оценки ожидаемого улучшения (Ур.7). Однако технически модель не параметризует $p(y)$, поскольку она вычисляется на основе наблюдаемых точек данных, которые сильно смещены в сторону низких значений из-за процесса оптимизации. В этой статье, следуя **Оценщику Парзена с древовидной структурой (Tree-structured Parzen Estimator)**, представленному Бергстра и др. [14], мы предлагаем формальное моделирование $p(x \mid y)$ и $p(y)$ вместо этого. Это делается путем построения двух параметрических плотностей, $g(x)$ и $b(x)$, которые вычисляются с использованием наблюдений со значением функции ниже и выше заданного порога соответственно. Разделяющий порог ν определяется как квантиль (т.е., среднее) смоделированных значений функции после наблюдения пространства гиперпараметров $\mathcal{H}$: $\nu = mean\{f(x_i), 1 \leq i \leq n\}$. Мы используем плотности $g(x)$ и $b(x)$, чтобы определить $p(x \mid y)$ как:

$$p(x \mid y) = g(x)I(y < \nu) + b(x)\big(1 - I(y < \nu)\big) \qquad (11)$$

где:

$$I = \begin{cases} 1 \text{ если важность}(x) < \text{важность}(x_{t+1}) \\ \text{см. раздел 2.3} \\ 0 \text{ иначе} \end{cases} \quad (12)$$

Мы принимаем формулировку EI, использованную в [14], заменяя их адаптивные оценщики Парзена на наш псевдо-апостериор из Ур. 11, т.е.:

$$\begin{aligned} EI_y(x) \quad &\equiv \int_{-\infty}^{+\infty} max(y^+ - y),0)p(y \mid x)\mathrm{d}y \\ &= \int_{-\infty}^{y^+} (y^+ - y),0)\frac{p(x \mid y)p(y)}{p(x)}\mathrm{d}y \end{aligned} \quad (13)$$

Ссылаясь на оценщик Парзена с древовидной структурой, мы определяем конфигурации как "плохие", если их наблюдаемая важность ниже определенного квантиля наблюдаемых значений функции. Поскольку γ – это корреляция целевых значений $\{y^{(i)}\}$, следовательно, $p(x)$ может быть представлена следующим образом:

$$\begin{aligned} p(x) &= \int_{\mathbb{R}} p(x \mid y)p(y)\mathrm{d}y \\ &= \gamma g(x) + (1-\gamma)b(x) \end{aligned} \quad (14)$$

более того,

$$\begin{aligned} \int_{-\infty}^{y^+} (y^+ - y)p(x \mid y)p(y)\mathrm{d}y &= g(x)\int_{-\infty}^{y^+} (y^+ - y)p(y)\mathrm{d}y \\ &= \gamma y^+ g(x) - g(x)\int_{-\infty}^{y^+} yp(y)\mathrm{d}y \end{aligned} \quad (15)$$

$$EI_{y^+}(x) = \frac{\gamma y^+ g(x) - g(x)\int_{-\infty}^{y^+} yp(y)\mathrm{d}y}{\gamma g(x) + (1-\gamma)b(x)} \quad (16)$$

Выражение $E_{I_y^+}(x)$ показывает, что для максимизации улучшения нам нужны конфигурации ГП H, обозначаемые как x, с высокой вероятностью выше $g(x)$ и под $b(x)$. Эта формулировка упрощает выбор многих кандидатов согласно γ и их оценку согласно $g(x)/b(x)$. На каждой итерации мы запускаем операцию argmax, которая ищет, возвращает ли алгоритм кандидата H^* (либо H^-, либо H^+ в зависимости от минимизации или максимизации функции потерь соответственно) с наибольшим EI, и, таким образом:

$$\arg\max EI_{H^*}(H) = \arg max \frac{g(H)}{b(H)} \quad (17)$$

Как правило, минимальный набор входных данных для функций приобретения в БО задается апостериорным прогнозом GP μ_t и σ_t. Для выполнения управления AF должна иметь возможность идентифицировать релевантную структуру, общую для целевых функций в $\mathcal{L}$. В нашей настройке это достигается путем расширения этого базового набора входных данных дополнительными признаками, которые позволяют AF оценивать места выборки. Поэтому, помимо среднего μ_t и дисперсии σ_t в потенциальных местах выборки, AF также получает важность входных данных, задаваемую fANOVA.

Может быть полезно отметить, что байесовская оптимизация является эффективным подходом для поиска оптимального решения задачи оптимизации с ограничением по времени из Ур. (3), однако, в

своей стандартной форме новые проблемы решаются с нуля. Лучшей альтернативой является "теплый старт" БО с конвейерами МО, которые, как ожидается, будут хорошо работать, как это делается в подходе k-ближайших наборов данных (KND) [1, 36]. Впоследствии анализ поведения кандидатного алгоритма (алгоритмов) на KND используется для начальной загрузки оптимизатора в качестве мета-управления.

5. Оценка

В этом разделе мы обсудим методологию оценки предлагаемого подхода для направления байесовской оптимизации алгоритмов МО. Мы изложим шаги, предпринятые для оценки эффективности предлагаемого подхода, и дадим подробное объяснение метрик, используемых для измерения его производительности. Кроме того, мы обсудим экспериментальную установку и наборы данных, использованные для проведения оценки.

5.1 Экспериментальная установка

Мы эмпирически оцениваем предлагаемый подход MGBO для оптимизации 08 соответствующих алгоритмов машинного обучения Scikit-Learn.

Dataset	Number of instances	Number of classes
Splice	3190	3
Numerai28.6	96320	2
Haberman	306	2
Eucalyptus	736	2
Abalone	4177	3
Scene	2407	2
Phoneme	5404	2
Bank8FM	8192	2

Table 3 List of the datasets used for the experiments; the names refer to the names on the OpenML platform

Эти алгоритмы вместе с исчерпывающим списком оптимизируемых гиперпараметров и пространством поиска приведены в Таблице 2. Пространство поиска ГП сокращено, чтобы сохранить вычисления управляемыми, а результаты интерпретируемыми.

Для наших экспериментов мы использовали 08 наборов данных, перечисленных в Таблице 3, с платформы OpenML4. Производительность валидации для байесовской оптимизации затем вычислялась с помощью стратифицированной 10-кратной кросс-валидации на обучающем наборе данных.

5.2 Экспериментальные результаты

Для нашей эмпирической оценки MGBO мы сравнили её с классической байесовской оптимизацией, использующей ожидаемое улучшение в качестве функции приобретения. Для каждого набора данных мы отслеживали реальное время выполнения. Рисунок 3 показывает улучшение, полученное с помощью нашего подхода с мета-управляемой оптимизацией.

Результаты, представленные на Рис. 3, демонстрируют, что мета-управляемая байесовская оптимизация с априорным управлением превосходит классическую БО на всех бенчмарк-наборах данных. Использование априорного управления позволяет MGBO достигать более эффективных результатов, используя информацию из ранее оцененных точек данных для направления поиска в перспективные области входного пространства. Особенно примечательно, что MGBO демонстрирует значительное ускорение на ранних фазах процесса оптимизации. В среднем ей требуется только 13 итераций для достижения того же уровня производительности, для достижения которого классической БО может потребоваться от 20 до 40 итераций. Это представляет ускорение примерно в три раза по сравнению с классической БО.

Улучшенная эффективность MGBO особенно выгодна в ситуациях, когда стоимость оценки целевой функции высока или задача оптимизации чувствительна ко времени. Сокращая количество итераций,

необходимых для достижения удовлетворительного решения, MGBO может помочь сэкономить вычислительные ресурсы и сократить время, необходимое для решения задач оптимизации. В целом, результаты подчеркивают преимущества включения априорных знаний в структуру байесовской оптимизации и предполагают, что MGBO является многообещающим подходом для решения сложных задач оптимизации более эффективным способом, чем классическая БО.

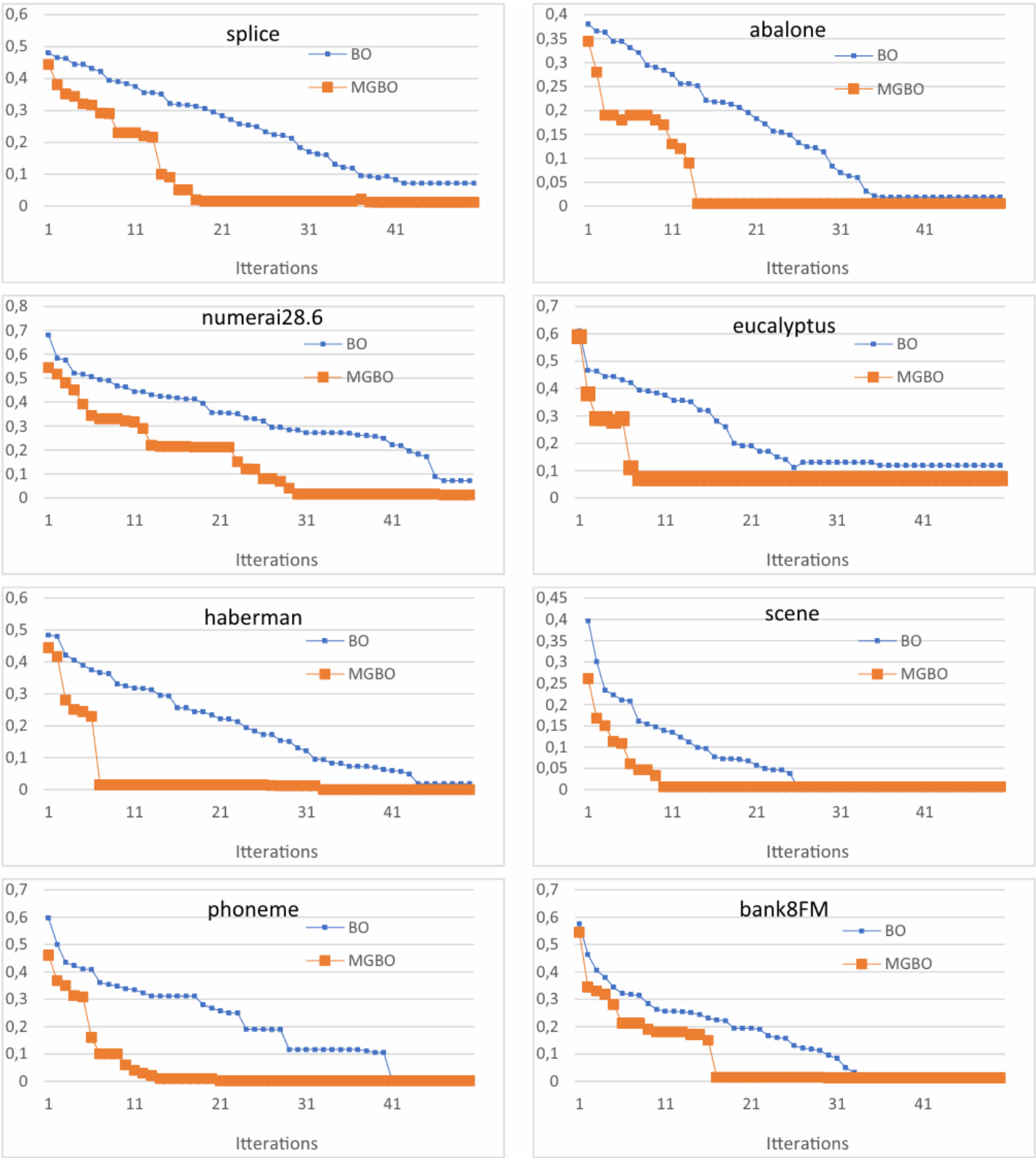

Fig.3 Convergence of tuning policies of the traditional BO and the proposed MGBO

6. Заключение

В последние годы байесовская оптимизация стала популярным подходом для настройки гиперпараметров в машинном обучении. Методы БО используют вероятностные модели для исследования пространства гиперпараметров алгоритма машинного обучения, направляемые наблюдаемой производительностью алгоритма на валидационном наборе. Однако байесовская

оптимизация может быть вычислительно затратной, особенно при работе с высокоразмерными пространствами поиска или сложными алгоритмами. Перенося априорные знания о том, какие части входного пространства могут дать наилучшую производительность из предыдущих прогонов оптимизации, мы разработали простой, но эффективный метод улучшения последовательной байесовской оптимизации на основе модели для гиперпараметров МО. Наш подход объединяет идеи как из метаобучения, так и из функционального дисперсионного анализа и включает инициализацию и направление БО конфигурациями, предложенными процедурой мета-управления. В этой работе мы показываем, как оптимизаторы "черного ящика" могут быть ускорены с использованием простых мета-знаний для направления исследования пространства поиска. Эксперименты, в которых участвовали 8 алгоритмов МО на 8 наборах данных, демонстрируют, что наш подход с мета-управляемой байесовской оптимизацией может находить хорошие конфигурации ГП в 2-3 раза быстрее, чем стандартная байесовская оптимизация. В будущей работе мы планируем изучить, как наш подход может быть использован для использования ещё большего количества знаний и руководства от метаобучения. Объединяя сильные стороны БО и метаобучения, мы считаем, что можем ещё больше улучшить эффективность и результативность настройки гиперпараметров в машинном обучении. В целом, наш подход имеет потенциал для обеспечения более быстрой и точной оптимизации гиперпараметров, что приведет к созданию лучших моделей машинного обучения и более эффективному использованию вычислительных ресурсов.

Список литературы

1. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Using meta-learning for automated algorithms selection and configuration: an experimental framework for industrial big data 9(1), 57 (2022). https://doi.org/10.1186/s40537-022-00612-4

2. Muñoz, M.A., Sun, Y., Kirley, M., Halgamane, S.K.: Algorithm selection for black-box continuous optimization problems: a survey on methods and challenges. Inf. Sci. 317, 224-245 (2015) https://doi.org/10.1016/j.ins.2015.05.010

3. Tighineanu, P., Skubch, K., Baireuther, P., Reiss, A., Berkenkamp, F., Vinogradska, J.: Transfer learning with gaussian processes for bayesian optimization. In: Proceedings of The 25th International Conference on Artificial Intelligence and Statistics. Proceedings of Machine Learning Research, vol. 151, pp. 6152-6181 (2022). https://proceedings.mlr.press/v151/tighineanu22a.html

4. Wu, J., Chen, X.-Y., Zhang, H., Xiong, L.-D., Lei, H., Deng, S.-H.: Hyperparameter optimization for machine learning models based on Bayesian optimization. J. Electron. Sci. Technol. 17(1), 26-40 (2019) https://doi.org/10.11989/JEST.1674-862X.80904120

5. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Towards big industrial data mining through explainable automated machine learning. Int. J. Adv. Manuf. Technol. 120(1), 1169-1188 (2022). https://doi.org/10.1007/s00170-022-08761-9

6. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M., Bourguin, G., Lewandowski, A.: Towards meta-learning based data analytics to better assist the domain experts in industry 4.0. In: Artificial Intelligence in Data and Big Data Processing, pp. 265-277 (2022). https://doi.org/10.1007/978-3-030-97610-1_22

7. Garouani, M., Hamlich, M., Ahmad, A., Bouneffa, M., Bourguin, G., Lewandowski, A.: Toward an automatic assistance framework for the selection and configuration of machine learning based data analytics solutions in industry 4.0. In: Proceedings of the 5th International Conference on Big Data and Internet of Things, pp. 3-15 (2022). https://doi.org/10.1007/978-3-031-07969-6_1

8. Hamed, O., Hamlich, M.: Improvised multi-robot cooperation strategy for hunting a dynamic target. In: 2020 International Symposium on Advanced Electrical and Communication Technologies (ISAEACT) (2020). https://doi.org/10.1109/isaeact50560.2020.9523684

9. Domhan, T., Springenberg, J.T., Hutter, F.: Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In: Proceedings of the 24th International Conference on Artificial Intelligence. IJCAI'15, pp. 3460-3468 (2015)

10. Volpp, M., Frhlich, L.P., Fischer, K., Doerr, A., Falkner, S., Hutter, F., Daniel, C.: Meta-learning acquisition functions for transfer learning in Bayesian optimization (2020) arXiv:1904.02642

11. Garouani, M., Garouani, M., Garouani, M., Ahmad, A., Bouneffa, M., Lewandowski, A., Bourguin, G., Hamlich, M.: Towards the automation of industrial data science: a meta-learning based approach, pp. 709–716 (2022). https://doi.org/10.5220/ 0010457107090716

12. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: AMLBID: an auto-explained automated machine learning tool for big indus trial data. SoftwareX 17, 100919 (2022) https://doi.org/10.1016/j.softx.2021.100919

13. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: Scalable meta-Bayesian based hyperparameters optimization for machine learning. In: Communications in Computer and Information Science, pp. 173–186 (2022). https://doi.org/10.1007/978-3-03120490-6_14

14. Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B.: Algorithms for hyper-parameter optimization. In: Proceedings of the 24th International Conference on Neural Information Processing Systems. NIPS'11, pp. 2546–2554, Red Hook, NY, USA (2011)

15. Garouani, M., Kharroubi, J.: Towards a new lexicon-based features vector for sentiment analysis: Application to moroccan Arabic tweets. In: Advances in Information, Communication and Cybersecurity, pp. 67–76 (2022). https://doi.org/10.1007/978-3-03091738-8_7

16. Garouani, M., Chrita, H., Kharroubi, J.: Sentiment analysis of Moroccan tweets using text mining. In: Digital Technologies an Applications. Lecture Notes in Computer Science, pp. 597–608 (2021). https://doi.org/10.1007/978-3-030-73882-2_54

17. Hamlich, M., Ramdani, M.: Applying the sac algorithm to extract the cardiologic indicators of anathlete's leve. Int. J. Intell. Inf. Syst.5, 23–27 (2016) https://doi.org/10.11648/j.ijiis.s.2016050301.13

18. Garouani, M.: Towards Efficient and Explainable Automated Machine Learning Pipelines Design : Application to Industry 4.0 Data. Theses, Université du Littoral Côte d'Opale; Université Hassan II (Casablanca, Maroc) (2022). https://theses.hal.science/tel03842609

19. Padierna,L.C.,Carpio,M.,Rojas,A.,Puga,H.,Baltazar,R.,Fraire, H.: Hyper-parameter tuning for support vector machines by estimation of distribution algorithms. In: Nature-Inspired Design ofHybrid Intelligent Systems, pp. 787–800 (2017). https://doi.org/10.1007/978-3-319-47054-2_53

20. Souza, A., Nardi, L., Oliveira, L.B., Olukotun, K., Lindauer, M., Hutter, F.: Bayesian optimization with a prior for the optimum. In:Machine Learning and Knowledge Discovery in Databases. Lecture Notes in Computer Science, pp. 265–296 (2021). https://doi.org/10.1007/978-3-030-86523-8_17

21. Kunjir, M.: Guided bayesian optimization to AutoTune memorybased analytics. In: 2019 IEEE 35th International Conference on Data Engineering Workshops, pp. 125–132 (2019). https://doi.org/10.1109/ICDEW.2019.00-22

22. Garouani,M.,Zaysa,K.:Leveragingtheautomatedmachinelearning for arabic opinion mining: a preliminary study on AutoML tools and comparison to human performance. In: Digital Technologies and Applications, pp. 163–171 (2022). https://doi.org/10.1007/978-3-031-02447-4_17

23. Priya,R.,Souza,B.F.,Rossi,A.L.D.,Carvalho:usinggeneticalgo rithms to improve prediction of execution times of ML tasks. In:Hybrid Artificial Intelligent Systems, pp. 196–207 (2012). https://doi.org/10.1007/978-3-642-28942-2_18

24. Makarova, A., Shen, H., Perrone, V., Klein, A., Faddoul, J.B., Krause, A., Seeger, M., Archambeau, C.: Automatic termination for hyperparameter optimization (2022). https://doi.org/10.48550/arXiv.2104.08166

25. Frazier, P.I.: Bayesian optimization. In: Recent Advances in Optimization and Modeling of Contemporary Problems, pp. 255–278 (2018). https://doi.org/10.1287/educ.2018.0188

26. Mockus, J., Tiesis, V., Zilinskas, A.: The application of Bayesian methods for seekingtheextremum.J.Abbr.:TowardsGlob.Optim. 2, 117–129 (2014)

27. Jones, D.R.: A taxonomy of global optimization methods based on response surfaces 21(4), 345-383 (2001). https://doi.org/10.1023/A:1012771025575

28. Srinivas, N., Krause, A., Kakade, S., Seeger, M.: Gaussian process optimization in the bandit setting: no regret and experimental design. In: Proceedings of the 27th International Conference on International Conference on Machine Learning, pp. 1015-1022 (2010)

29. Hernández-Lobato, J.M., Hoffman, M.W., Ghahramani, Z.: Predictive entropy search for efficient global optimization of black-box functions (2014). https://doi.org/10.48550/arXiv.1406.2541

30. Wu, J., Poloczek, M., Wilson, A.G., Frazier, P.I.: Bayesian optimization with gradients (2018). https://doi.org/10.48550/arXiv.1703.04389

31. Huang, J.Z.: Functional anova models for generalized regression. J. Multivar. Anal. 67(1), 49-71 (1998). https://doi.org/10.1006/jmva.1998.1753

32. Hutter, F., Hoos, H., Leyton-Brown, K.: An efficient approach for assessing hyperparameter importance. In: Xing, E.P., Jebara, T. (eds.) Proceedings of the 31st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 32, pp. 754-762. Beijing, China (2014)

33. Garouani, M., Ahmad, A., Bouneffa, M., Hamlich, M.: Autoencoder-knn meta-model based data characterization approach for an automated selection of ai algorithms. J. Big Data 10(14) (2023). https://doi.org/10.1186/s40537-023-00687-7

34. Rasmussen, C.E.: Gaussian processes in machine learning. Lecture Notes in Computer Science, pp. 63-71 (2004). https://doi.org/10.1007/978-3-540-28650-9_4

35. Wistuba, M., Schilling, N., Schmidt-Thieme, L.: Scalable gaussian process-based transfer surrogates for hyperparameter optimization 107, 43-78 (2018). https://doi.org/10.1007/s10994-017-5684-y

36. Feurer, M., Eggensperger, K., Falkner, S., Lindauer, M., Hutter, F.: Auto-Sklearn 2.0: hands-free AutoML via meta-learning (2020). https://doi.org/10.48550/ARXIV.2007.04074